

MATLAB Plotting Script: Nyquist Diagram

Chat transcript, 1–2 October 2026

You

(Screenshot of instructions, plus the existing MATLAB script.)

We are going to make a third plot now, also in the same window as before. Rescale the window to fit if necessary.

The third plot is the nyquist diagram. We only want the nyquist sometimes, and the other plots sometimes- so similar to outputting the plots with a true/false, let the user specify true/false for Time Domain, Frequency Domain, and Nyquist Diagram.

The nyquist diagram needs an input of Signal Frequency, and Sample Frequency (both in Hz, not kHz, or MHz, etc, the input is in Hz). let the user specify these with a 'signal' and 'source' variable that the user can define. If the user specifies that they want the nyquist diagram plotted with a 'true', use these frequencies to calculate the plot.

Only draw enough 'rungs' of the nyquist diagram as is necessary, plus a few for clarity. Attached is an image of how the nyquist diagram should look.

If only two plots are selected (any two), stack them one on top of the other. The order should be: time domain, frequency domain, nyquist. if all three plots are requested, stack the time and frequency domain on top of each other, with the nyquist diagram to their right. The nyquist diagram should be scaled to the height of the two other plots in this case.

Show less

Original script:

```
% Plots the DAT (time vs. voltage) and FFT (frequency vs. voltage) files
% for one experiment, both in the same figure window.
% Files: 9_17_2026jamesXY.txt, X = experiment number, Y = 'dat' or 'fft'.
% Each file: tab-delimited, 1 header line, 2 columns.
clear; clc; close all;

%% ===== USER SETTINGS =====
expNum = 11; % experiment number (integer)
dataFolder = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Matlab'; % folder containing the .txt files

% ---- OPTION B: specify file names manually instead ----
useManualNames = false;
FFT = "fft.txt";
DAT = "dat.txt";

% ---- DAT plot axis limits (override) ----
% Leave as [] to autoscale to the min/max of the data.
% Otherwise use [min max], e.g. timeLim = [0 0.005]; voltLim = [-3 3];
timeLim = [0 0.01]; % X axis (time, s)
voltLim = [-5 5]; % Y axis (voltage, V)
%% =====

%% Build file paths
if useManualNames
    datFile = fullfile(dataFolder, DAT);
    fftFile = fullfile(dataFolder, FFT);
else
    prefix = sprintf('9_17_2026james%d', expNum);
```

```

    datFile = fullfile(dataFolder, [prefix 'dat.txt']);
    fftFile = fullfile(dataFolder, [prefix 'fft.txt']);
end

if ~isfile(datFile), error('DAT file not found: %s', datFile); end
if ~isfile(fftFile), error('FFT file not found: %s', fftFile); end

%% Read data (1 header line, tab-delimited)
datData = readmatrix(datFile, 'NumHeaderLines', 1, 'Delimiter', '\t');
fftData = readmatrix(fftFile, 'NumHeaderLines', 1, 'Delimiter', '\t');

t = datData(:,1); % time [s]
vt = datData(:,2); % voltage [V]
f = fftData(:,1); % frequency [Hz]
vf = fftData(:,2); % voltage [V]

%% Plot both in one figure
figure('Name', sprintf('Experiment %d', expNum), 'NumberTitle', 'off');
tl = tiledlayout(2, 1, 'TileSpacing', 'compact');
title(tl, sprintf('Experiment %d', expNum));

% --- Plot 1: Time vs. Voltage (DAT) ---
nexttile;
plot(t, vt, 'LineWidth', 1.2);
xlabel('Time (s)');
ylabel('Voltage (V)');
title('Time vs. Voltage');
grid on;
xlim(getLimits(t, timeLim));
ylim(getLimits(vt, voltLim));

% --- Plot 2: Frequency vs. Voltage (FFT) ---
nexttile;
plot(f, vf, 'LineWidth', 1.2);
xlabel('Frequency (Hz)');
ylabel('Voltage (V)');
title('Frequency vs. Voltage');
grid on;
axis tight;

%% Local function: autoscale to data min/max unless user override is given
function lim = getLimits(x, userLim)
    if isempty(userLim)
        lim = [min(x) max(x)];
        if lim(1) == lim(2)
            lim = lim + [-1 1];
        end
    else
        lim = userLim;
    end
end
end

```

Claude

I've updated the script and pushed it to `claude/adoring-pasteur-6vq2a7` as `plotExperiment.m`. There's no MATLAB in this environment, so I haven't run it. Check the figure on your machine.

The Nyquist example image didn't come through. The only image I got was the screenshot of your instructions. I drew the standard folding diagram (the Figliola style) instead. If yours looks different, send the image again and I'll match it.

New settings:

```

showTimeDomain = true; % Time vs. Voltage
showFreqDomain = true; % Frequency vs. Voltage
showNyquist     = true; % Nyquist diagram

signal = 1200; % signal frequency [Hz]
source = 1000; % sample frequency [Hz]

```

Layout:

- **One plot:** it fills the window.

- **Two plots (any two):** stacked top to bottom in the order time, frequency, Nyquist. The window height grows to fit.
- **All three:** time on top of frequency on the left, with the Nyquist diagram on the right as tall as both. The window widens to 1400×700.

How the Nyquist diagram works:

- **Shape:** the frequency axis is folded back and forth every $f_N = \text{source}/2$. Each "rung" covers one f_N -wide band and has the frequency in Hz labelled at both ends.
- **Rungs:** it draws only the rungs needed to reach the signal frequency, plus 2 more for clarity.
- **Signal:** a red dot on its rung. A dashed line drops from it to the first rung, where a blue square marks the aliased frequency f_a .
- **Title:** shows f_s , f_N and f_a .

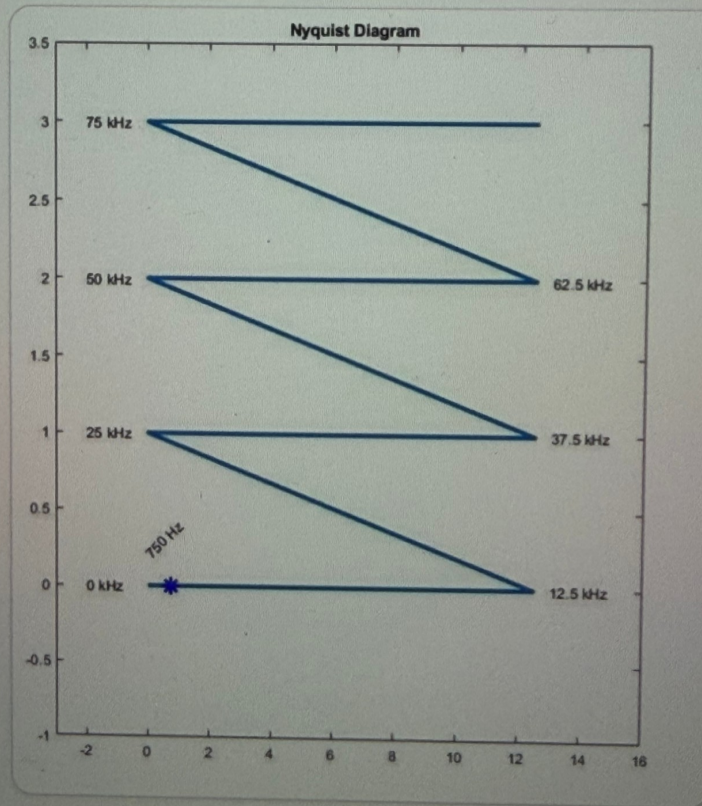
Other changes:

- The script only loads the DAT or FFT file if the plot that uses it is turned on. For example, with only the Nyquist diagram on, it won't fail when the data files are missing.
- It stops with an error if every plot is off, if the sample frequency is 0 or less, or if the signal frequency is negative.

If the signal frequency is much higher than the sample frequency, there will be a lot of rungs, so the labels will get crowded.

You

change the code to make the nyquist diagram look like:



a bigger change.

the same window as before. Rescale the window

Claude

I redrew the Nyquist diagram to match your picture and pushed it to the same branch. I still can't run MATLAB here, so it's untested; run it once on your machine to check it.

What the diagram looks like now:

- **Shape:** one zig-zag line. Each flat rung runs left to right from 0 to f_N (half the sample frequency), and a diagonal runs back up to the left end of the next rung.
- **Labels:** frequencies sit at both ends of each rung (0 / 12.5 kHz, 25 / 37.5 kHz, and so on). The script picks Hz, kHz or MHz based on the sample frequency, and the x-axis uses the same unit.
- **Signal:** a blue * with the frequency written at a 45° angle next to it, like the "750 Hz" in your picture.
- **Rungs:** it draws the rungs needed to reach the signal frequency, plus 3 more for clarity. That gives the 4 rungs in your picture.
- **Axes:** the box and y-axis numbers are shown as in your picture, and the title is "Nyquist Diagram".

One addition your picture doesn't show: if the signal frequency is above f_N , the star lands on a higher rung or a diagonal. In that case a red dashed line drops from it to the bottom rung, and a red dot there is labelled with the aliased frequency.

I changed the default settings to `signal = 750` and `source = 25000`, so a first run should reproduce your example.

You

give me the code to copy and paste

Claude

Here's the full script. Paste it into a single .m file in MATLAB. Functions at the end of a script need MATLAB R2016b or newer, and tiledlayout needs R2019b or newer.

(Full script — identical to the final version reproduced at the end of this document.)

You

it says Error: File: JFinalCode2.m Line: 126 Column: 16

Function 'getLimits' has already been declared within this scope. fix it and give me the code

Claude

This error means the file has two functions named getLimits. The code I sent declares it only once (at line 107), and your error points to line 126. So the new code was probably pasted in without fully replacing the old code, which left two copies of the function in JFinalCode2.m.

To fix it:

- Open JFinalCode2.m.
- Press **Ctrl+A** to select everything, then **Delete**, so the file is empty.
- Paste the code below and save.

When it's pasted correctly, the file is exactly 203 lines long. There should be one getLimits (around line 107) and one plotNyquist (around line 125). If you still get an error after that, send me the new message and the line it points to.

Final script:

```
% Plots the DAT (time vs. voltage) and FFT (frequency vs. voltage) files
% for one experiment, plus an optional Nyquist (folding) diagram, all in
% the same figure window.
% Files: 9_17_2026jamesXY.txt, X = experiment number, Y = 'dat' or 'fft'.
% Each file: tab-delimited, 1 header line, 2 columns.
clear; clc; close all;

%% ===== USER SETTINGS =====
expNum = 11; % experiment number (integer)
dataFolder = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Matlab'; % folder containing the .txt files

% ---- OPTION B: specify file names manually instead ----
useManualNames = false;
FFT = "fft.txt";
DAT = "dat.txt";

% ---- Which plots to show (true/false) ----
showTimeDomain = true; % Time vs. Voltage (DAT file)
showFreqDomain = true; % Frequency vs. Voltage (FFT file)
showNyquist = true; % Nyquist (folding) diagram
```

```

% ---- Nyquist diagram inputs (both in Hz) ----
signal = 750;           % signal frequency [Hz]
source = 25000;         % sample frequency [Hz]

% ---- DAT plot axis limits (override) ----
% Leave as [] to autoscale to the min/max of the data.
% Otherwise use [min max], e.g. timeLim = [0 0.005]; voltLim = [-3 3];
timeLim = [0 0.01];     % X axis (time, s)
voltLim = [-5 5];       % Y axis (voltage, V)
%% =====

nPlots = showTimeDomain + showFreqDomain + showNyquist;
if nPlots == 0, error('No plots selected. Set at least one of the show* flags to true.');
```

end

```

%% Build file paths
if useManualNames
    datFile = fullfile(dataFolder, DAT);
    fftFile = fullfile(dataFolder, FFT);
else
    prefix = sprintf('9_17_2026james%d', expNum);
    datFile = fullfile(dataFolder, [prefix 'dat.txt']);
    fftFile = fullfile(dataFolder, [prefix 'fft.txt']);
end

%% Read data (1 header line, tab-delimited) -- only the files that are needed
if showTimeDomain
    if ~isfile(datFile), error('DAT file not found: %s', datFile); end
    datData = readmatrix(datFile, 'NumHeaderLines', 1, 'Delimiter', '\t');
    t = datData(:,1); % time [s]
    vt = datData(:,2); % voltage [V]
end
if showFreqDomain
    if ~isfile(fftFile), error('FFT file not found: %s', fftFile); end
    fftData = readmatrix(fftFile, 'NumHeaderLines', 1, 'Delimiter', '\t');
    f = fftData(:,1); % frequency [Hz]
    vf = fftData(:,2); % voltage [V]
end

%% Figure and layout
% 1 plot -> single tile
% 2 plots -> stacked vertically (order: time, frequency, nyquist)
% 3 plots -> time over frequency on the left, nyquist on the right spanning both rows
fig = figure('Name', sprintf('Experiment %d', expNum), 'NumberTitle', 'off');
if nPlots == 3
    fig.Position = [100 100 1400 700];
    tl = tiledlayout(2, 2, 'TileSpacing', 'compact');
    tiles = {1, 3, 2}; % tile index for time, freq, nyquist
    nyqSpan = [2 1];
else
    fig.Position = [100 100 800 350*nPlots];
    tl = tiledlayout(nPlots, 1, 'TileSpacing', 'compact');
    tiles = num2cell(cumsum([showTimeDomain showFreqDomain showNyquist]));
    nyqSpan = [1 1];
end
title(tl, sprintf('Experiment %d', expNum));

% --- Plot 1: Time vs. Voltage (DAT) ---
if showTimeDomain
    nexttile(tiles{1});
    plot(t, vt, 'LineWidth', 1.2);
    xlabel('Time (s)');
    ylabel('Voltage (V)');
    title('Time vs. Voltage');
    grid on;
    xlim(getLimits(t, timeLim));
    ylim(getLimits(vt, voltLim));
end

% --- Plot 2: Frequency vs. Voltage (FFT) ---
if showFreqDomain
    nexttile(tiles{2});
    plot(f, vf, 'LineWidth', 1.2);
    xlabel('Frequency (Hz)');
    ylabel('Voltage (V)');
    title('Frequency vs. Voltage');
    grid on;
    axis tight;
end

```

```

% --- Plot 3: Nyquist (folding) diagram ---
if showNyquist
    ax = nexttile(tiles{3}, nyqSpan);
    plotNyquist(ax, signal, source);
end

%% Local function: autoscale to data min/max unless user override is given
function lim = getLimits(x, userLim)
    if isempty(userLim)
        lim = [min(x) max(x)];
        if lim(1) == lim(2)
            lim = lim + [-1 1];
        end
    else
        lim = userLim;
    end
end

%% Local function: Nyquist folding diagram
% Zig-zag of the frequency axis folded at multiples of fN = fs/2:
% level m (y = m) is a horizontal rung from 2m*fN (left) to (2m+1)*fN (right),
% and a diagonal runs from (2m+1)*fN back up-left to 2(m+1)*fN on the next level.
% A frequency on any segment aliases to the frequency directly below it on
% the bottom rung (0 .. fN).
function plotNyquist(ax, fSig, fs)
    if fs <= 0, error('Sample frequency (source) must be > 0 Hz.');
```

```

    end
    if fSig < 0, error('Signal frequency (signal) must be >= 0 Hz.');
```

```

    end

    fN = fs/2;
    [uScale, uName] = pickUnits(fs); % display units for labels / x axis

    % Which band (k) the signal is in, and where along it (r = 0..1)
    k = floor(fSig/fN);
    r = fSig/fN - k;
    if k > 0 && r == 0 % exactly on a fold: use end of previous band
        k = k - 1;
        r = 1;
    end
    m = floor(k/2); % level the signal's band starts on
    if mod(k, 2) == 0 % horizontal rung, left -> right
        xSig = r*fN;
        ySig = m;
    else % diagonal, right -> up-left
        xSig = (1 - r)*fN;
        ySig = m + r;
    end
    fAlias = xSig;

    nExtra = 3; % extra levels drawn above the signal for clarity
    nLevels = m + 1 + nExtra;

    % Zig-zag path: (0,0) -> (fN,0) -> (0,1) -> (fN,1) -> ...
    xp = repmat([0 fN], 1, nLevels);
    yp = repelem(0:nLevels-1, 2);

    hold(ax, 'on');
    plot(ax, xp/uScale, yp, '-', 'LineWidth', 2, 'Color', [0 0.3 0.6]);

    % Level labels: left end = 2m*fN, right end = (2m+1)*fN
    dx = 0.08*fN/uScale;
    for lv = 0:nLevels-1
        text(ax, -dx, lv, fmtFreq(2*lv*fN, uScale, uName), ...
            'HorizontalAlignment', 'right', 'VerticalAlignment', 'middle');
        text(ax, fN/uScale + dx, lv, fmtFreq((2*lv+1)*fN, uScale, uName), ...
            'HorizontalAlignment', 'left', 'VerticalAlignment', 'middle');
    end

    % Signal frequency marker
    plot(ax, xSig/uScale, ySig, '*', 'Color', 'b', 'MarkerSize', 12, 'LineWidth', 1.5);
    text(ax, xSig/uScale, ySig + 0.08, [' ' fmtFreq(fSig, 1, 'Hz')], ...
        'Rotation', 45, 'Color', 'b', 'HorizontalAlignment', 'left', ...
        'VerticalAlignment', 'bottom');

    % If the signal is above fN, project it down to the bottom rung (alias)
    if fSig > fN
        plot(ax, [xSig xSig]/uScale, [ySig 0], 'r--', 'LineWidth', 1.2);
        plot(ax, fAlias/uScale, 0, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 7);
        text(ax, fAlias/uScale, -0.08, [' ' fmtFreq(fAlias, 1, 'Hz') ' (alias)'], ...
            'Rotation', 45, 'Color', 'r', 'HorizontalAlignment', 'left', ...
            'VerticalAlignment', 'bottom');
    end
end

```

```

        'Rotation', -45, 'Color', 'r', 'HorizontalAlignment', 'left', ...
        'VerticalAlignment', 'top');
    end
    hold(ax, 'off');

    xlim(ax, [-0.25*fN 1.3*fN]/uScale);
    ylim(ax, [-1, nLevels - 0.5]);
    box(ax, 'on');
    title(ax, 'Nyquist Diagram');
end

function [scale, name] = pickUnits(f)
    if f >= 1e6
        scale = 1e6; name = 'MHz';
    elseif f >= 1e3
        scale = 1e3; name = 'kHz';
    else
        scale = 1; name = 'Hz';
    end
end

function s = fmtFreq(f, scale, name)
    s = sprintf('%g %s', f/scale, name);
end

```

You

make this chat a pdf